

Of course. Here is a practical, step-by-step guide to completing **Step 1** of your research. This will give you the raw material needed for the subsequent AI-focused steps.

Executing Step 1: Making Your "Shopping List" (Collecting the Key Data)

Objective: To create four separate, clean data files (as CSV or Excel sheets) containing the foundational information for your research.

Sub-Step 1.1: Create Your Plant List

Goal: Curate a focused list of well-documented South Indian medicinal plants traditionally used for diabetes.

Action:

1. Start with a small, manageable list of 5-10 plants. You can always add more later.
2. Create a file named plants.csv.
3. In this file, create two columns: Plant_ID and Plant_Name.

Example plants.csv file:

Plant_ID	Plant_Name
P001	Gymnema sylvestre (Sirkurinjai)
P002	Azadirachta indica (Neem)
P003	Momordica charantia (Bitter Gourd)
P004	Tinospora cordifolia (Seenthil)
P005	Syzygium cumini (Naaval)

Sub-Step 1.2: Find the Chemicals in Each Plant

Goal: For each plant in your list, find its known chemical constituents (phytochemicals).

Recommended Tool: IMPPAT (Indian Medicinal Plants, Phytochemistry and Therapeutics) Database. It is specifically designed for this purpose.

Action:

1. Go to the IMPPAT website.
2. In the search bar, type the name of one of your plants (e.g., "Gymnema sylvestre").
3. Click on the plant name in the search results.
4. On the plant's page, look for a section called "**Phytochemistry**" or "**Phytochemicals**". This will list all the chemicals found in that plant.
5. Create a new file named phytochemicals.csv.

- For each chemical you find, record its name and, most importantly, its **PubChem CID** (Compound ID). The CID is a unique number that will act as a universal identifier for that chemical.
- Also record which plant it came from.

Example phytochemicals.csv file:

PubChem_CID	Chemical_Name	Plant_Source
160539	Gymnemic acid	P001
5280863	Gurmarin	P001
6325996	Azadirachtin	P002
14982	Nimbin	P002
5281585	Charantin	P003

(...and so on for every chemical in every plant)

Sub-Step 1.3: Identify Your Protein Targets

Goal: Select 2-3 key proteins in the human body that are known targets for anti-diabetic drugs.

Recommended Tool: UniProt (Universal Protein Resource). This is the gold standard for protein information.

Action:

- Let's choose two highly-validated targets for Type 2 Diabetes:
 - Dipeptidyl peptidase 4 (DPP-4)**
 - Alpha-glucosidase**
- Create a file named proteins.csv.
- For each protein, go to the UniProt website and search for its name. Find the entry for *Homo sapiens* (Human) and record its **UniProt Accession ID**. This is the unique identifier for the protein.

Example proteins.csv file:

UniProt_ID	Protein_Name
P27487	Dipeptidyl peptidase 4 (DPP-4)
P10253	Alpha-glucosidase (Maltase-glucoamylase)

Sub-Step 1.4: Find Known Connections (Your "Answer Key")

Goal: Find chemicals that are already proven to interact with your chosen protein targets. This data is essential for training the AI.

Recommended Tool: BindingDB. It's a database of known binding interactions between drugs/compounds and proteins.

Action:

1. Go to the BindingDB website.
2. Click on "Browse" and then search by "Target Name".
3. In the search box, enter the name of your first protein target: Dipeptidyl peptidase 4.
4. The website will return a list of many small molecules (ligands) that are known to bind to DPP-4.
5. Create a final file named interactions.csv.
6. For each interaction you find, you need to record the **PubChem CID** of the chemical and the **UniProt ID** of the protein.
7. Repeat this process for your second protein, "Alpha-glucosidase".

Example interactions.csv file:

PubChem_CID	UniProt_ID
6918547	P27487
444133	P27487
5311108	P27487
2038	P10253

(...this list will likely have hundreds or thousands of entries)

End Result of Step 1

You have now successfully completed the data collection phase. You should have four simple files:

1. **plants.csv:** Your list of local plants.
2. **phytochemicals.csv:** The chemicals found in those plants.
3. **proteins.csv:** The diabetes targets you are investigating.
4. **interactions.csv:** Your crucial "answer key" of known chemical-protein connections.

The **IDs** (PubChem_CID and UniProt_ID) are the magic "glue" that will allow you to connect all these separate files in the next step. You are now perfectly set up for **Step 2: Connect the Dots.**

Excellent. Let's dive into the full, detailed procedure for **Step 2**. This is where we transition from having separate, flat data files into a living, connected network—the brain for our AI.

Executing Step 2: Connect the Dots (Build a Simple Network)

Objective: To write a Python script that reads the four CSV files from Step 1 and uses the NetworkX library to construct a single graph object. This graph will contain all the plants, chemicals, and proteins as "nodes" and their relationships as "edges."

Prerequisites: Setting Up Your Environment

Before you write any code, make sure you have the necessary tools installed and your files are organized.

1. **Python:** Ensure you have Python 3 installed on your computer.
2. **Required Libraries:** Open your terminal or command prompt and install the necessary libraries using pip. pandas is for reading your CSV files, networkx is for building the graph, and matplotlib is for seeing it.

Generated bash

```
pip install pandas networkx matplotlib
```

3. **File Organization:** Create a new folder for your project. Place your Python script and the four CSV files (plants.csv, phytochemicals.csv, proteins.csv, interactions.csv) all inside this same folder. This makes file paths simple.
-

The Detailed Procedure: From CSVs to a Graph

We will build the script piece by piece, explaining what each part does.

Part 2.1: Import Libraries and Load Data

First, we need to import the libraries we installed and load our four CSV files into memory using the pandas library. Pandas reads them into data structures called "DataFrames," which are essentially powerful, programmable spreadsheets.

Generated python

```
# Import the necessary libraries
import pandas as pd
import networkx as nx
import matplotlib.pyplot as plt

# --- Load all data from CSV files into Pandas DataFrames ---
```

```

print("Step 1: Loading data from CSV files...")

try:
    plants_df = pd.read_csv('plants.csv')
    chemicals_df = pd.read_csv('phytochemicals.csv')
    proteins_df = pd.read_csv('proteins.csv')
    interactions_df = pd.read_csv('interactions.csv')
    print("Data loaded successfully!")
except FileNotFoundError as e:
    print(f"Error: {e}. Make sure all CSV files are in the same folder as the script.")
    exit()

```

IGNORE_WHEN_COPYING_START

content_copy download

Use code [with caution](#). Python

IGNORE_WHEN_COPYING_END

Explanation:

- We import the three libraries we need.
- We use a try...except block, which is good practice. It will give a clean error message if it can't find one of the CSV files.
- `pd.read_csv()` reads each file into its own DataFrame (e.g., `plants_df`).

Part 2.2: Initialize the Graph and Add Nodes

Now, we create an empty graph object using NetworkX. Then, we'll loop through our DataFrames and add each unique plant, chemical, and protein as a "node" to our graph. It's crucial to add attributes (like type and name) to each node so we can identify them later.

Generated python

```

# --- Initialize a new graph ---
G = nx.Graph()
print("\nStep 2: Creating graph and adding nodes...")

# Add Plant nodes
for index, row in plants_df.iterrows():
    G.add_node(row['Plant_ID'], type='plant', name=row['Plant_Name'])

# Add Chemical nodes
# We use PubChem_CID as the unique identifier for chemicals
for index, row in chemicals_df.iterrows():
    # Make sure to convert CID to string to be consistent
    node_id = str(row['PubChem_CID'])
    G.add_node(node_id, type='chemical', name=row['Chemical_Name'])

# Add Protein nodes
for index, row in proteins_df.iterrows():
    G.add_node(row['UniProt_ID'], type='protein', name=row['Protein_Name'])

print(f"Added {G.number_of_nodes()} total nodes.")

```

IGNORE_WHEN_COPYING_START

content_copy download

Use code [with caution](#). Python

IGNORE_WHEN_COPYING_END

Explanation:

- `G = nx.Graph()` creates our empty network.
- We loop through each row of the `plants_df` DataFrame. For each plant, we call `G.add_node()`.
 - The first argument (`row['Plant_ID']`) is the **unique ID** for the node.
 - The following arguments (`type='plant', name=row['Plant_Name']`) are **attributes**. These are labels that store extra information about the node. This is extremely useful.
- We repeat this process for chemicals (using `PubChem_CID` as the ID) and proteins (using `UniProt_ID` as the ID). We convert the CID to a string `str()` to avoid any potential data type issues.

Part 2.3: Add Edges (The Connections)

A graph is useless without edges to connect the nodes. We'll now loop through our relationship files (`phytochemicals.csv` and `interactions.csv`) to create the links.

Generated python

```
print("\nStep 3: Adding edges to connect the nodes...")

# Add edges: Chemical 'is_found_in' Plant
for index, row in chemicals_df.iterrows():
    chemical_id = str(row['PubChem_CID'])
    plant_id = row['Plant_Source']
    G.add_edge(chemical_id, plant_id, relation='is_found_in')

# Add edges: Chemical 'targets' Protein
for index, row in interactions_df.iterrows():
    chemical_id = str(row['PubChem_CID'])
    protein_id = row['UniProt_ID']
    # We only add the edge if both the chemical and protein already exist
    as nodes
    if G.has_node(chemical_id) and G.has_node(protein_id):
        G.add_edge(chemical_id, protein_id, relation='targets')

print(f"Added {G.number_of_edges()} total edges.")
```

IGNORE_WHEN_COPYING_START

content_copy download

Use code [with caution](#). Python

IGNORE_WHEN_COPYING_END

Explanation:

- We loop through `chemicals_df`. For each row, we create an edge between the chemical's ID and the plant's ID. We give this edge an attribute `relation='is_found_in'`.
- We do the same for `interactions_df`, creating an edge between a chemical and a protein with the attribute `relation='targets'`.

- The if G.has_node(...) check is an important safety measure. It ensures we don't try to create an edge to a node that doesn't exist in our graph (e.g., if an interaction in interactions.csv involves a chemical not from our plant list).

Part 2.4: Verification and Visualization

Finally, let's print some basic statistics to verify our graph was built correctly and then create a simple visualization to see what we've made.

Generated python

```
print("\n--- Graph Construction Complete! ---")
print(f"Total Nodes: {G.number_of_nodes()}")
print(f"Total Edges: {G.number_of_edges()}")

# --- Optional: Visualize the graph ---
print("\nStep 4: Creating a visualization of the graph (this may take a
moment)...")

# Define colors for each node type for better visualization
color_map = []
for node in G:
    if G.nodes[node]['type'] == 'plant':
        color_map.append('green')
    elif G.nodes[node]['type'] == 'protein':
        color_map.append('red')
    else:
        color_map.append('skyblue')

# Draw the graph
plt.figure(figsize=(12, 12))
nx.draw(G, node_color=color_map, with_labels=False, node_size=50)
plt.title("Visualization of the Phytochemical Knowledge Graph")
plt.show()

print("\nStep 2 is complete. The graph object 'G' is now ready for Step
3.")
```**Explanation:**
* We print the total number of nodes and edges, which is a great sanity
check.
* We create a `color_map` list. We loop through every node in the graph
and assign a color based on its `type` attribute. This makes the
visualization much clearer.
* `nx.draw()` is the function that draws the graph. `plt.show()` displays
it.
* **Note:** For a very large graph, this visualization will look like a
"hairball." But for our starting dataset, it provides a powerful and
satisfying confirmation that our network is connected as expected.

The Complete Script

Just copy and paste the code below into a Python file (e.g.,
`build_graph.py`), place it in the same folder as your CSVs, and run it.

```python
#
=====
==
```

```

# Ph.D. Research - Step 2: Build Knowledge Graph
# This script reads data from CSV files and constructs a NetworkX graph.
#
=====

# Import the necessary libraries
import pandas as pd
import networkx as nx
import matplotlib.pyplot as plt

# --- Step 1: Load all data from CSV files into Pandas DataFrames ---
print("Step 1: Loading data from CSV files...")
try:
    plants_df = pd.read_csv('plants.csv')
    chemicals_df = pd.read_csv('phytochemicals.csv')
    proteins_df = pd.read_csv('proteins.csv')
    interactions_df = pd.read_csv('interactions.csv')
    print("Data loaded successfully!")
except FileNotFoundError as e:
    print(f"Error: {e}. Make sure all CSV files are in the same folder as the script.")
    exit()

# --- Step 2: Initialize a new graph and add nodes ---
G = nx.Graph()
print("\nStep 2: Creating graph and adding nodes...")

# Add Plant nodes
for index, row in plants_df.iterrows():
    G.add_node(row['Plant_ID'], type='plant', name=row['Plant_Name'])

# Add Chemical nodes (using PubChem_CID as the unique identifier)
for index, row in chemicals_df.iterrows():
    node_id = str(row['PubChem_CID']) # Ensure CID is a string
    G.add_node(node_id, type='chemical', name=row['Chemical_Name'])

# Add Protein nodes
for index, row in proteins_df.iterrows():
    G.add_node(row['UniProt_ID'], type='protein', name=row['Protein_Name'])

print(f"Added {G.number_of_nodes()} total nodes.")

# --- Step 3: Add edges to connect the nodes ---
print("\nStep 3: Adding edges to connect the nodes...")

# Add edges: Chemical 'is_found_in' Plant
for index, row in chemicals_df.iterrows():
    chemical_id = str(row['PubChem_CID'])
    plant_id = row['Plant_Source']
    G.add_edge(chemical_id, plant_id, relation='is_found_in')

# Add edges: Chemical 'targets' Protein (from the known interactions file)
for index, row in interactions_df.iterrows():
    chemical_id = str(row['PubChem_CID'])
    protein_id = row['UniProt_ID']
    # Safety check: only add edge if both nodes exist in our graph
    if G.has_node(chemical_id) and G.has_node(protein_id):
        G.add_edge(chemical_id, protein_id, relation='targets')

```



```

print(f"Added {G.number_of_edges()} total edges.")
print("\n--- Graph Construction Complete! ---")
print(f"Total Nodes in Graph: {G.number_of_nodes()}")
print(f"Total Edges in Graph: {G.number_of_edges()}")

# --- Step 4 (Optional but Recommended): Visualize the graph ---
print("\nStep 4: Creating a visualization of the graph (this may take a
moment)...")

# Define colors for each node type for a clear visualization
color_map = []
for node in G:
    if G.nodes[node]['type'] == 'plant':
        color_map.append('green')
    elif G.nodes[node]['type'] == 'protein':
        color_map.append('red')
    else: # chemical
        color_map.append('skyblue')

# Draw the graph using matplotlib
plt.figure(figsize=(12, 12))
pos = nx.spring_layout(G, k=0.15, iterations=20) # A layout for better
spacing
nx.draw(pos, node_color=color_map, with_labels=False, node_size=50)
plt.title("Visualization of the Phytochemical Knowledge Graph")
plt.show()

print("\nStep 2 is complete. The graph object 'G' is now created and ready
for Step 3 (AI Training).")

```

IGNORE_WHEN_COPYING_START
 content_copy download
 Use code [with caution](#). Python
 IGNORE_WHEN_COPYING_END

After running this script, you will have successfully transformed your separate data lists into a single, rich, interconnected graph object G. This is the exact input required for the AI model in Step 3.

Of course. Let's move on to the most exciting part: **Step 3**. This is where the "intelligence" comes in. We will take the graph G we built in Step 2 and use it to train a basic AI model to understand the patterns of drug-target interactions.

Executing Step 3: Train Your "Detective" (Use a Basic AI Model)

Objective: To write a Python script that:

1. Loads the graph G from Step 2.
 2. Prepares the graph data for a machine learning model.
 3. Defines a simple Graph Neural Network (GNN) using PyTorch Geometric.
 4. Trains the GNN to predict links between chemicals and proteins.
 5. Saves the trained "detective" model for use in the final step.
-

Prerequisites: Setting Up Your AI Environment

This step requires specialized deep learning libraries.

1. **PyTorch:** This is the core deep learning framework. Go to the [PyTorch website](https://pytorch.org/) and select the correct installation command for your system (CPU is fine for this starting example).
2. **PyTorch Geometric (PyG):** This library is built on top of PyTorch and makes working with GNNs much easier. After installing PyTorch, install the PyG libraries.

Generated bash

```
pip install torch_geometric
# Depending on your PyTorch/CUDA version, you might need additional
packages.
# The PyG website has a helpful installation wizard: https://pytorch-
geometric.readthedocs.io/en/latest/install/installation.html
pip install pyg_lib torch_scatter torch_sparse -f
https://data.pyg.org/whl/torch-2.1.0+cpu.html
# ^^^ IMPORTANT: Adjust the URL based on your PyTorch and CPU/GPU
version!
```

3. **File Organization:** You need the `build_graph.py` script from Step 2 to be runnable, or you can save the graph object G to a file and load it here. For simplicity, we'll combine the scripts: this script will first build the graph from the CSVs and then immediately train on it.
-

The Detailed Procedure: From Graph to Trained Model

This script will be longer, so we'll break it down into logical parts.

Part 3.1: Data Preparation for the AI

A GNN can't work with a NetworkX object directly. It needs numerical data in a specific format called **tensors**. We need to convert our graph into this format.

1. **Node Features (x):** The AI needs a numerical description for each node. The simplest way is **one-hot encoding**. We'll create a unique vector for each node.
2. **Edge Index (edge_index):** The AI needs a list of all connections.
3. **Training Labels (edge_label and edge_label_index):** We need to give the AI examples of "good" links (positive examples) and "bad" links (negative examples).
 - o **Positive Edges:** The known targets relationships from our interactions.csv.
 - o **Negative Edges:** We will randomly sample pairs of chemicals and proteins that are *not* known to be connected. This is crucial for teaching the model what *not* to predict.

Part 3.2: Defining the GNN Model

We will define our "detective" using PyTorch. It will be a simple GCN (Graph Convolutional Network) with two main tasks:

1. `encode()`: Learn a rich numerical vector (an "embedding") for every node in the graph. This embedding will capture information about the node's position and neighborhood.
2. `decode()`: Take the embeddings of two nodes (a chemical and a protein) and calculate a score representing the probability that they are connected.

Part 3.3: The Training Loop

This is the "learning" process. We will repeat these steps for a set number of "epochs" (cycles):

1. Pass the data through the model to get predictions.
2. Compare the model's predictions to the true labels (our positive/negative examples).
3. Calculate the "error" or "loss".
4. Use an "optimizer" to slightly adjust the model's internal parameters to reduce the error on the next cycle.

The Complete Script (train_model.py)

Create a new Python file named `train_model.py` and paste the following code into it. Read the comments carefully to understand each line.

Generated python

```
#
=====
==
# Ph.D. Research - Step 3: Train the AI Model (GNN)
# This script builds the graph, prepares it for PyTorch Geometric,
# defines a GNN model, trains it for link prediction, and saves the result.
```

```

#
=====

import pandas as pd
import networkx as nx
import torch
from torch_geometric.data import Data
from torch_geometric.nn import GCNConv
from torch_geometric.utils import negative_sampling
import torch.nn.functional as F

# --- Part 0: Re-run Graph Construction (from Step 2) ---
# In a real project, you'd save the graph and load it, but for simplicity,
# we just rebuild it here.
def build_graph():
    print("Building graph from CSVs...")
    plants_df = pd.read_csv('plants.csv')
    chemicals_df = pd.read_csv('phytochemicals.csv')
    proteins_df = pd.read_csv('proteins.csv')
    interactions_df = pd.read_csv('interactions.csv')

    G = nx.Graph()
    # Add nodes with attributes
    for _, row in plants_df.iterrows(): G.add_node(row['Plant_ID'],
type='plant', name=row['Plant_Name'])
    for _, row in chemicals_df.iterrows():
G.add_node(str(row['PubChem_CID']), type='chemical',
name=row['Chemical_Name'])
    for _, row in proteins_df.iterrows(): G.add_node(row['UniProt_ID'],
type='protein', name=row['Protein_Name'])

    # Add edges with attributes
    for _, row in chemicals_df.iterrows():
G.add_edge(str(row['PubChem_CID']), row['Plant_Source'],
relation='is_found_in')
    for _, row in interactions_df.iterrows():
        if G.has_node(str(row['PubChem_CID'])) and
G.has_node(row['UniProt_ID']):
            G.add_edge(str(row['PubChem_CID']), row['UniProt_ID'],
relation='targets')

    return G

G = build_graph()
print(f"Graph built with {G.number_of_nodes()} nodes and
{G.number_of_edges()} edges.")

# --- Part 1: Prepare Data for PyTorch Geometric ---
print("\n--- Preparing data for PyTorch Geometric ---")

# Create a mapping from node ID to an integer index
node_list = list(G.nodes())
node_to_int = {node: i for i, node in enumerate(node_list)}

# Create the edge_index tensor
# This describes all connections in the graph in the format PyG needs
edge_list = []
for u, v in G.edges():
    edge_list.append([node_to_int[u], node_to_int[v]])

```

```

    edge_list.append([node_to_int[v], node_to_int[u]]) # Add edges in both
directions for an undirected graph
edge_index = torch.tensor(edge_list, dtype=torch.long).t().contiguous()

# Create node features (x) using one-hot encoding
# This gives each node a unique numerical vector
num_nodes = G.number_of_nodes()
x = F.one_hot(torch.arange(0, num_nodes), num_classes=num_nodes).float()

# Create the PyG Data object
data = Data(x=x, edge_index=edge_index)

# Identify the edges for our training task (chemical-protein interactions)
interaction_edges = []
for u, v, attrs in G.edges(data=True):
    if attrs.get('relation') == 'targets':
        interaction_edges.append([node_to_int[u], node_to_int[v]])
train_pos_edge_index = torch.tensor(interaction_edges,
dtype=torch.long).t().contiguous()

# Sample negative edges (pairs that are not connected)
# This is crucial for teaching the model what a "bad" link looks like
num_neg_samples = train_pos_edge_index.size(1) # Match the number of
positive samples
train_neg_edge_index = negative_sampling(
    edge_index=data.edge_index,
    num_nodes=data.num_nodes,
    num_neg_samples=num_neg_samples
)

print(f"Prepared PyG data object. Positive examples:
{train_pos_edge_index.size(1)}, Negative examples:
{train_neg_edge_index.size(1)}")

# --- Part 2: Define the GNN Model ---
print("\n--- Defining the GNN Model ---")

class GCNLinkPredictor(torch.nn.Module):
    def __init__(self, in_channels, hidden_channels, out_channels):
        super().__init__()
        self.conv1 = GCNConv(in_channels, hidden_channels)
        self.conv2 = GCNConv(hidden_channels, out_channels)

    def encode(self, x, edge_index):
        # Learn node embeddings
        x = self.conv1(x, edge_index).relu()
        return self.conv2(x, edge_index)

    def decode(self, z, edge_label_index):
        # Calculate a score for a given pair of nodes (an edge)
        # We use a simple dot product
        return (z[edge_label_index[0]] * z[edge_label_index[1]]).sum(dim=-
1)

# Initialize the model
# in_channels = number of features per node (which is num_nodes because of
one-hot)
# hidden_channels = an intermediate size, a hyperparameter to tune
# out_channels = the size of the final embedding vector for each node
model = GCNLinkPredictor(data.num_nodes, 128, 64)

```

```

optimizer = torch.optim.Adam(params=model.parameters(), lr=0.01)

# --- Part 3: The Training Loop ---
print("\n--- Starting Model Training ---")

def train():
    model.train()
    optimizer.zero_grad()

    # 1. Get node embeddings from the encoder
    z = model.encode(data.x, data.edge_index)

    # 2. Get scores for positive and negative edges
    pos_logits = model.decode(z, train_pos_edge_index)
    neg_logits = model.decode(z, train_neg_edge_index)

    # 3. Combine them and create labels
    logits = torch.cat([pos_logits, neg_logits])
    labels = torch.cat([torch.ones(pos_logits.size(0)),
    torch.zeros(neg_logits.size(0))])

    # 4. Calculate the loss (how wrong the model is)
    loss = F.binary_cross_entropy_with_logits(logits, labels)

    # 5. Backpropagate the error and update the model
    loss.backward()
    optimizer.step()

    return loss

for epoch in range(1, 201):
    loss = train()
    if epoch % 10 == 0:
        print(f"Epoch: {epoch:03d}, Loss: {loss:.4f}")

print("--- Training Complete ---")

# --- Part 4: Save the Trained Model ---
print("\n--- Saving the trained model ---")
# We save the trained state of the model, and also the node mapping
# which is essential for decoding the results later.
torch.save(model.state_dict(), 'trained_model.pth')
torch.save(node_to_int, 'node_to_int_mapping.pth')
torch.save(node_list, 'node_list.pth') # Save the node list as well

print("\nModel saved as 'trained_model.pth'.")
print("Node mapping saved as 'node_to_int_mapping.pth'.")
print("Step 3 is complete. You are now ready for the final prediction
step!")

```

IGNORE_WHEN_COPYING_START
 content_copy download
 Use code [with caution](#). Python
 IGNORE_WHEN_COPYING_END

How to Run It and What to Expect

1. **Save the Code:** Save the script as `train_model.py` in the same folder as your four CSV files.
2. **Run from Terminal:** Open your terminal, navigate to that folder, and run the script:

Generated bash

```
python train_model.py
```

IGNORE_WHEN_COPYING_START

content_copy download

Use code [with caution](#). Bash

IGNORE_WHEN_COPYING_END

3. **Watch the Output:** You will see a series of print statements:
 - First, it will confirm the graph is built.
 - Next, it will show that the PyG data is prepared.
 - Then, you will see the training progress, with the "Loss" value printed every 10 epochs. **You want to see this number generally decreasing**, as it means the model is learning.
 - Finally, it will confirm that the model and mappings have been saved.

After the script finishes, you will have three new files in your folder:

- `trained_model.pth`: This is your "detective"—the trained AI, ready to be used.
- `node_to_int_mapping.pth`: A dictionary that remembers which integer corresponds to which node ID (e.g., 'P001': 0, '6325996': 1, etc.).
- `node_list.pth`: A list of the nodes in the order they were indexed.

You have successfully taught an AI to recognize the patterns in your data. You are now fully prepared for **Step 4**, where we will use this trained model to find new, exciting scientific clues.

Excellent. We have arrived at the final and most rewarding part of the process: **Step 4**.

Here, we will use the "detective" model we trained in Step 3 to analyze our entire knowledge map and identify new, previously unknown connections. This is where the scientific discovery happens.

Executing Step 4: Ask the Detective for New Clues (Get Predictions)

Objective: To write a Python script that:

1. Loads our saved AI model and the necessary data mappings.
 2. Generates a list of all possible "chemical-protein" pairs that were *not* in the original training data.
 3. Uses the trained model to predict the probability of an interaction for each pair.
 4. Ranks the results and presents a clean, readable list of the top potential discoveries.
-

Prerequisites

Your environment should already be set up from Step 3. The only requirement is to have the output files from the previous step in the same folder:

- `trained_model.pth` (the AI model itself)
 - `node_to_int_mapping.pth` (the dictionary mapping node names to numbers)
 - `node_list.pth` (the ordered list of all node names)
 - Your four original CSV files (needed to rebuild the graph for context).
-

The Detailed Procedure: From Trained Model to New Discoveries

This script will load everything, generate predictions, and then translate the numerical output back into meaningful names for our final report.

Part 4.1: Load Everything Back

We'll start by loading the model, the mappings, and rebuilding the graph. It is critical that the graph structure and node ordering are identical to how they were during training.

Part 4.2: Generate Candidate Edges for Prediction

We can't just ask the model to "find things." We have to give it a specific list of pairs to evaluate. Our strategy is:

1. Identify all nodes that are 'chemicals' and all nodes that are 'proteins'.
2. Create every possible (chemical, protein) pair. This is our "candidate list."

3. Crucially, we must remove any pairs from this list that we already know are connected (i.e., the ones from interactions.csv), because we are only interested in finding *new* interactions.

Part 4.3: Predict and Rank

With our list of new candidate pairs, we'll run them through the model's decode function. This will give us a raw score for each potential link. We will then convert these scores into probabilities (from 0 to 1) and sort them to find the most likely new discoveries.

Part 4.4: Present the Results

A list of numbers is not a result. The final step is to take the top predictions and display them clearly, showing the chemical's name, the plant it comes from, the protein it's predicted to target, and the model's confidence score.

The Complete Script (predict.py)

Create a new Python file named predict.py. Paste in the following code. The comments explain each action in detail.

Generated python

```
#
=====
==
# Ph.D. Research - Step 4: Get Predictions from the Trained Model
# This script loads the trained GNN, generates candidate links,
# predicts their probabilities, and displays the top results.
#
=====
==

import pandas as pd
import networkx as nx
import torch
import torch.nn.functional as F
from torch_geometric.data import Data
from torch_geometric.nn import GCNConv

# --- Part 0: Re-define the GNN Model Class ---
# We must define the exact same model architecture as during training
# so we can load the saved weights into it.
class GCNLinkPredictor(torch.nn.Module):
    def __init__(self, in_channels, hidden_channels, out_channels):
        super().__init__()
        self.conv1 = GCNConv(in_channels, hidden_channels)
        self.conv2 = GCNConv(hidden_channels, out_channels)

    def encode(self, x, edge_index):
        x = self.conv1(x, edge_index).relu()
        return self.conv2(x, edge_index)

    def decode(self, z, edge_label_index):
```

```

        return (z[edge_label_index[0]] * z[edge_label_index[1]]).sum(dim=-
1)

# --- Part 1: Load All Saved Components ---
print("--- Step 1: Loading trained model and data mappings ---")
# Load the mappings first
node_to_int = torch.load('node_to_int_mapping.pth')
node_list = torch.load('node_list.pth')
int_to_node = {v: k for k, v in node_to_int.items()} # Create reverse
mapping

# Load the graph structure to get node features and edge index
# (This is the same data preparation as in Step 3)
G = nx.read_gpickle('graph.gpickle') # Assumes you saved the graph from
Step 2/3
num_nodes = G.number_of_nodes()
x = F.one_hot(torch.arange(0, num_nodes), num_classes=num_nodes).float()
edge_list = []
for u, v in G.edges():
    edge_list.append([node_to_int[u], node_to_int[v]])
    edge_list.append([node_to_int[v], node_to_int[u]])
edge_index = torch.tensor(edge_list, dtype=torch.long).t().contiguous()
data = Data(x=x, edge_index=edge_index)

# Initialize a new model with the same architecture
model = GCNLinkPredictor(data.num_nodes, 128, 64)
# Load the saved weights from training
model.load_state_dict(torch.load('trained_model.pth'))
model.eval() # Set the model to evaluation mode
print("Model and data loaded successfully.")

# --- Part 2: Generate Candidate Edges for Prediction ---
print("\n--- Step 2: Generating candidate interactions to predict ---")

# Get lists of integer indices for all chemicals and proteins
chemical_indices = [node_to_int[node] for node, attrs in G.nodes(data=True)
if attrs['type'] == 'chemical']
protein_indices = [node_to_int[node] for node, attrs in G.nodes(data=True)
if attrs['type'] == 'protein']

# Create all possible (chemical, protein) pairs
candidate_pairs = []
for chem_idx in chemical_indices:
    for prot_idx in protein_indices:
        candidate_pairs.append([chem_idx, prot_idx])
candidate_edge_index = torch.tensor(candidate_pairs,
dtype=torch.long).t().contiguous()

# Filter out edges that already exist in the graph
known_edges_set = set()
for u, v in G.edges():
    u_idx, v_idx = node_to_int[u], node_to_int[v]
    known_edges_set.add(tuple(sorted((u_idx, v_idx))))

filtered_candidate_pairs = []
for u, v in candidate_pairs:
    if tuple(sorted((u, v))) not in known_edges_set:
        filtered_candidate_pairs.append([u, v])

if not filtered_candidate_pairs:
    print("No new interactions to predict!")

```

```

exit()

prediction_edge_index = torch.tensor(filtered_candidate_pairs,
dtype=torch.long).t().contiguous()
print(f"Found {prediction_edge_index.size(1)} new potential interactions to
evaluate.")

# --- Part 3: Predict Scores for New Interactions ---
print("\n--- Step 3: Using the trained model to predict scores ---")

with torch.no_grad(): # We don't need to calculate gradients for prediction
    # First, get the final node embeddings from our trained model
    z = model.encode(data.x, data.edge_index)
    # Then, get the scores for our candidate edges
    scores = model.decode(z, prediction_edge_index)
    # Convert raw scores to probabilities using the sigmoid function
    probabilities = torch.sigmoid(scores)

# --- Part 4: Rank and Display the Top Results ---
print("\n--- Step 4: Top Predicted Novel Interactions ---")

results = []
for i in range(prediction_edge_index.size(1)):
    u, v = prediction_edge_index[:, i]
    prob = probabilities[i].item()

    # Get the human-readable names
    chemical_node_id = int_to_node[u.item()]
    protein_node_id = int_to_node[v.item()]

    chemical_name = G.nodes[chemical_node_id].get('name', 'N/A')
    protein_name = G.nodes[protein_node_id].get('name', 'N/A')

    # Find the plant source of the chemical
    plant_source_name = 'Unknown'
    for neighbor in G.neighbors(chemical_node_id):
        if G.nodes[neighbor].get('type') == 'plant':
            plant_source_name = G.nodes[neighbor].get('name', 'N/A')
            break

    results.append({
        "Chemical": chemical_name,
        "Source Plant": plant_source_name,
        "Predicted Target": protein_name,
        "Confidence Score": prob
    })

# Sort the results by confidence score in descending order
sorted_results = sorted(results, key=lambda x: x['Confidence Score'],
reverse=True)

# Print the top 15 predictions
print("\n=====
=====")
print("
TOP 15 NEW PREDICTIONS
")
print("=====
=====")
for i, result in enumerate(sorted_results[:15]):
    print(f"Rank {i+1}:")

```

```

    print(f" - Chemical: {result['Chemical']} (from {result['Source
Plant']})")
    print(f" - Predicted to Target: {result['Predicted Target']}")
    print(f" - Model Confidence: {result['Confidence Score']:.2%}\n") #
Formatted as percentage

if not sorted_results:
    print("No new interactions were found to rank.")

```

How to Run It and Interpret the Output

1. **Save the Code:** Save it as predict.py in the same folder as all your other files.
2. **Run from Terminal:**

Generated bash

```
python predict.py
```

IGNORE_WHEN_COPYING_START

content_copy download

Use code [with caution](#). Bash

IGNORE_WHEN_COPYING_END

3. **Analyze the Output:** The script will print its progress and then display a final, clean list. This list is the primary result of your entire project.

Example Output:

Generated code

```

=====
=====
                                TOP 15 NEW PREDICTIONS
=====
=====
Rank 1:
  - Chemical: Gymnemic acid (from Gymnema sylvestre (Sirkurinjai))
  - Predicted to Target: Dipeptidyl peptidase 4 (DPP-4)
  - Model Confidence: 96.45%

Rank 2:
  - Chemical: Nimbin (from Azadirachta indica (Neem))
  - Predicted to Target: Alpha-glucosidase
  - Model Confidence: 91.82%

... and so on

```

IGNORE_WHEN_COPYING_START

content_copy download

Use code [with caution](#).

IGNORE_WHEN_COPYING_END

This output provides a clear, prioritized list of hypotheses. A result like "Rank 1" suggests that, based on all the patterns the AI learned, it is highly confident that "Gymnemic acid" might inhibit the "DPP-4" protein, a key anti-diabetic mechanism. This gives a biologist or pharmacologist a specific, high-potential experiment to run in the lab.

Congratulations! You have now completed the full, end-to-end workflow of a sophisticated computational drug discovery project.

Yes, absolutely. The workflow we designed is a fantastic and complete project. However, there are many other ways to approach this problem or ask different questions using the same data.

Think of these as alternative research paths you could take, depending on your interests and comfort level.

Here are three distinct "other" approaches:

Alternative 1: The Simpler, Non-Graph Approach (Classical Machine Learning)

This method ignores the *network connections* and focuses only on the properties of the chemicals themselves. It's less holistic but much simpler to implement.

Research Question: Can we predict if a chemical is anti-diabetic based purely on its molecular structure?

Simple Method:

1. **Gather Data:** Create just two lists:
 - **List A (Positives):** All the chemicals KNOWN to work against your target proteins (from interactions.csv).
 - **List B (Negatives):** A random sample of other chemicals from your plants that are NOT known to work.
2. **Generate "Fingerprints":** For every chemical in both lists, use a library like **RDKit** in Python to convert its structure into a "molecular fingerprint." This is a long vector of 0s and 1s that represents the presence or absence of specific structural features.
3. **Train a Simple Classifier:** Instead of a GNN, you now have a standard classification problem. You can use a much simpler, classic AI model:
 - **Model Choice: Random Forest or XGBoost.** These are powerful, fast, and easy to interpret.
 - **Training:** You feed the model the fingerprints (X data) and the labels (y data: 1 for "works," 0 for "doesn't work").
4. **Predict:** Give the model the fingerprints of all the other plant chemicals it has never seen before and ask it to predict whether they are a "1" or a "0".

Why choose this?

- **Simplicity:** This is a classic, well-understood machine learning workflow. It avoids the complexity of PyTorch Geometric and graph theory.
 - **Great Baseline:** This is an excellent "baseline model." In your Ph.D., you could do this *first* and then show how your more complex GNN model (from our original plan) performs even better, proving the value of using the network information.
-

Alternative 2: The Language-Focused Approach (NLP + Knowledge Graph)

This method enriches your graph by "reading" scientific literature to find information that isn't in structured databases.

Research Question: Can we build a more powerful predictive model by automatically extracting knowledge from thousands of scientific papers?

Simple Method:

1. **Build Your Basic Graph:** Complete Step 1 and 2 from our original plan.
2. **Automated Literature Review:** Use a biomedical NLP model (like **BioBERT** or **SciBERT**) to process the abstracts of thousands of research papers from PubMed that mention your plants or chemicals.
3. **Extract New Relationships:** Program the model to find sentences that suggest relationships, like:
 - "We found that *Azadirachtin* **reduces** blood glucose..."
 - "*Curcumin* **showed an inhibitory effect** on *DPP-4*..."
4. **Add to Your Graph:** Add these newly found relationships as new edges in your NetworkX graph. Your graph is now much richer, containing both database facts and literature evidence.
5. **Train the GNN:** Now, proceed with Step 3 (training the GNN) on this new, super-charged graph.

Why choose this?

- **High Novelty:** This is a very modern and impressive approach. Combining NLP with GNNs is a cutting-edge area of AI research.
 - **Richer Data:** You can uncover evidence and connections that are too recent or obscure to be in a formal database, potentially leading to more interesting discoveries.
-

Alternative 3: The "Big Picture" Approach (Community Detection & Plant Ranking)

This method changes the question. Instead of finding the best *chemical*, it tries to find the best *plant* or *groups* of chemicals.

Research Question: Which plants in our list are the most promising overall? And are there groups of chemicals that seem to work together?

Simple Method:

1. **Build Your Graph:** Complete Step 1 and 2 from our original plan.
2. **Find Chemical Communities:**
 - **Tool:** Use a community detection algorithm built into NetworkX (like the **Louvain algorithm**).
 - **Action:** Run the algorithm on your graph. It will automatically group nodes that are more connected to each other than to the rest of the graph.

- **Result:** You might find a "community" that contains 3 chemicals from different plants that all target the same protein. This suggests a common mechanism or a shared structural feature worth investigating.
3. **Rank the Most Important Plants:**
- **Tool:** Use a graph centrality algorithm like **PageRank** (the original algorithm Google used for ranking webpages).
 - **Action:** Run PageRank on your graph. It will assign an "importance score" to every node.
 - **Result:** You can then rank your *plant* nodes by their importance score. A plant with a high score is likely one that contains many chemicals connected to known anti-diabetic pathways, making it a high-priority candidate for further study.

Why choose this?

- **System-Level Insights:** This provides a "zoomed-out" view, generating hypotheses about systems and groups rather than single interactions.
- **Algorithmically Simpler:** These algorithms (Louvain, PageRank) are often single function calls in NetworkX, requiring less complex setup than a deep learning model.

Summary Table

Approach	Primary Goal	Complexity	Key Technology	Best For a Researcher Who...
Original Plan (GNN)	Find new chemical-protein interactions.	High	Graph Neural Networks	Wants to do a state-of-the-art deep learning project.
Alternative 1	Predict if a single chemical is active.	Low	Random Forest / RDKit	Wants a simpler, faster starting point or a solid baseline.
Alternative 2	Find interactions using papers + databases .	Very High	NLP (BERT) + GNN	Is ambitious and interested in language models and AI.
Alternative 3	Find the best plants or groups of chemicals .	Medium	Graph Algorithms (PageRank)	Is interested in systems biology and network analysis.